Predict the price of the Uber ride from a given pickup point to the agreed drop-off location. Perform following tasks:

1. Pre-process the dataset.
2. Identify outliers.
3. Check the correlation.
4. Implement linear regression and random forest regression models.
5. Evaluate the models and compare their respective scores like R2, RMSE, etc.

```
#Importing the required libraries
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

```
#importing the dataset
df  = pd.read_csv("uber.csv")
```

1. Pre-process the dataset.

```
df.head()
```

	fare_amount	pickup_longitude	pickup_latitude	dropoff_longitude	dropoff_latitude	passenger_count	hour	day	month	year
0	7.5	-73.999817	40.738354	-73.999512	40.723217	1.0	19	7	5	2015
1	7.7	-73.994355	40.728225	-73.994710	40.750325	1.0	20	17	7	2009
2	12.9	-74.005043	40.740770	-73.962565	40.772647	1.0	21	24	8	2009
3	5.3	-73.976124	40.790844	-73.965316	40.803349	3.0	8	26	6	2009
4	16.0	-73.929786	40.744085	-73.973082	40.761247	3.5	17	28	8	2014

```
df.info() #To get the required information of the dataset
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200000 entries, 0 to 199999
Data columns (total 9 columns):
 #   Column             Non-Null Count   Dtype  
---  ------             --------------   -----  
 0   Unnamed: 0         200000 non-null  int64  
 1   key                200000 non-null  object 
 2   fare_amount        200000 non-null  float64
 3   pickup_datetime    200000 non-null  object 
 4   pickup_longitude   200000 non-null  float64
 5   pickup_latitude    200000 non-null  float64
 6   dropoff_longitude  199999 non-null  float64
 7   dropoff_latitude   199999 non-null  float64
 8   passenger_count    200000 non-null  int64  
dtypes: float64(5), int64(2), object(2)
memory usage: 13.7+ MB
```

```
df.columns #TO get number of columns in the dataset
```

```
Index(['Unnamed: 0', 'key', 'fare_amount', 'pickup_datetime',
       'pickup_longitude', 'pickup_latitude', 'dropoff_longitude',
       'dropoff_latitude', 'passenger_count'],
      dtype='object')
```

```
df = df.drop(['Unnamed: 0', 'key'], axis= 1) #To drop unnamed column as it isn't required
```

```
df.head()
```

	fare_amount	pickup_datetime	pickup_longitude	pickup_latitude	dropoff_longitude	dropoff_latitude	passenger_count
0	7.5	2015-05-07 19:52:06 UTC	-73.999817	40.738354	-73.999512	40.723217	1
1	7.7	2009-07-17 20:04:56 UTC	-73.994355	40.728225	-73.994710	40.750325	1
2	12.9	2009-08-24 21:45:00 UTC	-74.005043	40.740770	-73.962565	40.772647	1
3	5.3	2009-06-26					

```
df.shape #To get the total (Rows,Columns)
```

```
(200000, 7)
```

```
df.dtypes #To get the type of each column
```

	0
fare_amount	float64
pickup_datetime	object
pickup_longitude	float64
pickup_latitude	float64
dropoff_longitude	float64
dropoff_latitude	float64
passenger_count	int64

dtype: object

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200000 entries, 0 to 199999
Data columns (total 7 columns):
 #   Column             Non-Null Count   Dtype  
---  ------             --------------   -----  
 0   fare_amount        200000 non-null  float64
 1   pickup_datetime    200000 non-null  object 
 2   pickup_longitude   200000 non-null  float64
 3   pickup_latitude    200000 non-null  float64
 4   dropoff_longitude  199999 non-null  float64
 5   dropoff_latitude   199999 non-null  float64
 6   passenger_count    200000 non-null  int64  
dtypes: float64(5), int64(1), object(1)
memory usage: 10.7+ MB
```

```
df.describe() #To get statistics of each columns
```

	fare_amount	pickup_longitude	pickup_latitude	dropoff_longitude	dropoff_latitude	passenger_count
count	200000.000000	200000.000000	200000.000000	199999.000000	199999.000000	200000.000000
mean	11.359955	-72.527638	39.935885	-72.525292	39.923890	1.684535
std	9.901776	11.437787	7.720539	13.117408	6.794829	1.385997
min	-52.000000	-1340.648410	-74.015515	-3356.666300	-881.985513	0.000000
25%	6.000000	-73.992065	40.734796	-73.991407	40.733823	1.000000
50%	8.500000	-73.981823	40.752592	-73.980093	40.753042	1.000000
75%	12.500000	-73.967154	40.767158	-73.963658	40.768001	2.000000
max	499.000000	57.418457	1644.421482	1153.572603	872.697628	208.000000

bold text##### Filling Missing values

```
df.isnull().sum()
```

	0
fare_amount	0
pickup_datetime	0
pickup_longitude	0
pickup_latitude	0
dropoff_longitude	1
dropoff_latitude	1
passenger_count	0

dtype: int64

```
df['dropoff_latitude'].fillna(value=df['dropoff_latitude'].mean(),inplace = True)
df['dropoff_longitude'].fillna(value=df['dropoff_longitude'].median(),inplace = True)
```

```
/tmp/ipython-input-1690886834.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chair
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are set

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = c


  df['dropoff_latitude'].fillna(value=df['dropoff_latitude'].mean(),inplace = True)
/tmp/ipython-input-1690886834.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chair
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are set

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = c


  df['dropoff_longitude'].fillna(value=df['dropoff_longitude'].median(),inplace = True)
```

```
df.isnull().sum()
```

	0
fare_amount	0
pickup_datetime	0
pickup_longitude	0
pickup_latitude	0
dropoff_longitude	0
dropoff_latitude	0
passenger_count	0

dtype: int64

```
df.dtypes
```

	0
fare_amount	float64
pickup_datetime	object
pickup_longitude	float64
pickup_latitude	float64
dropoff_longitude	float64
dropoff_latitude	float64
passenger_count	int64

dtype: object

Column pickup_datetime is in wrong format (Object). Convert it to DateTime Format

```
df.pickup_datetime = pd.to_datetime(df.pickup_datetime, errors='coerce')
```

```
df.dtypes
```

	0
fare_amount	float64
pickup_datetime	datetime64[ns, UTC]
pickup_longitude	float64
pickup_latitude	float64
dropoff_longitude	float64
dropoff_latitude	float64
passenger_count	int64

dtype: object

To segregate each time of date and time

```
df= df.assign(hour = df.pickup_datetime.dt.hour,
              day= df.pickup_datetime.dt.day,
              month = df.pickup_datetime.dt.month,
```

```
            year = df.pickup_datetime.dt.year,
            dayofweek = df.pickup_datetime.dt.dayofweek)
```

```
df.head()
```

	fare_amount	pickup_datetime	pickup_longitude	pickup_latitude	dropoff_longitude	dropoff_latitude	passenger_count	hour
0	7.5	2015-05-07 19:52:06+00:00	-73.999817	40.738354	-73.999512	40.723217	1	19
1	7.7	2009-07-17 20:04:56+00:00	-73.994355	40.728225	-73.994710	40.750325	1	20
2	12.9	2009-08-24 21:45:00+00:00	-74.005043	40.740770	-73.962565	40.772647	1	21
		2009-06-26						

```
# drop the column 'pickup_daetime' using drop()
# 'axis = 1' drops the specified column

df = df.drop('pickup_datetime',axis=1)
```

```
df.head()
```

	fare_amount	pickup_longitude	pickup_latitude	dropoff_longitude	dropoff_latitude	passenger_count	hour	day	month	year
0	7.5	-73.999817	40.738354	-73.999512	40.723217	1	19	7	5	2015
1	7.7	-73.994355	40.728225	-73.994710	40.750325	1	20	17	7	2009
2	12.9	-74.005043	40.740770	-73.962565	40.772647	1	21	24	8	2009
3	5.3	-73.976124	40.790844	-73.965316	40.803349	3	8	26	6	2009
4	16.0	-73.925023	40.744085	-73.973082	40.761247	5	17	28	8	2014

```
df.dtypes
```

	0
fare_amount	float64
pickup_longitude	float64
pickup_latitude	float64
dropoff_longitude	float64
dropoff_latitude	float64
passenger_count	int64
hour	int32
day	int32
month	int32
year	int32
dayofweek	int32

dtype: object

Checking outliers and filling them

```
df.plot(kind = "box",subplots = True,layout = (7,2),figsize=(15,20)) #Boxplot to check the outliers
```

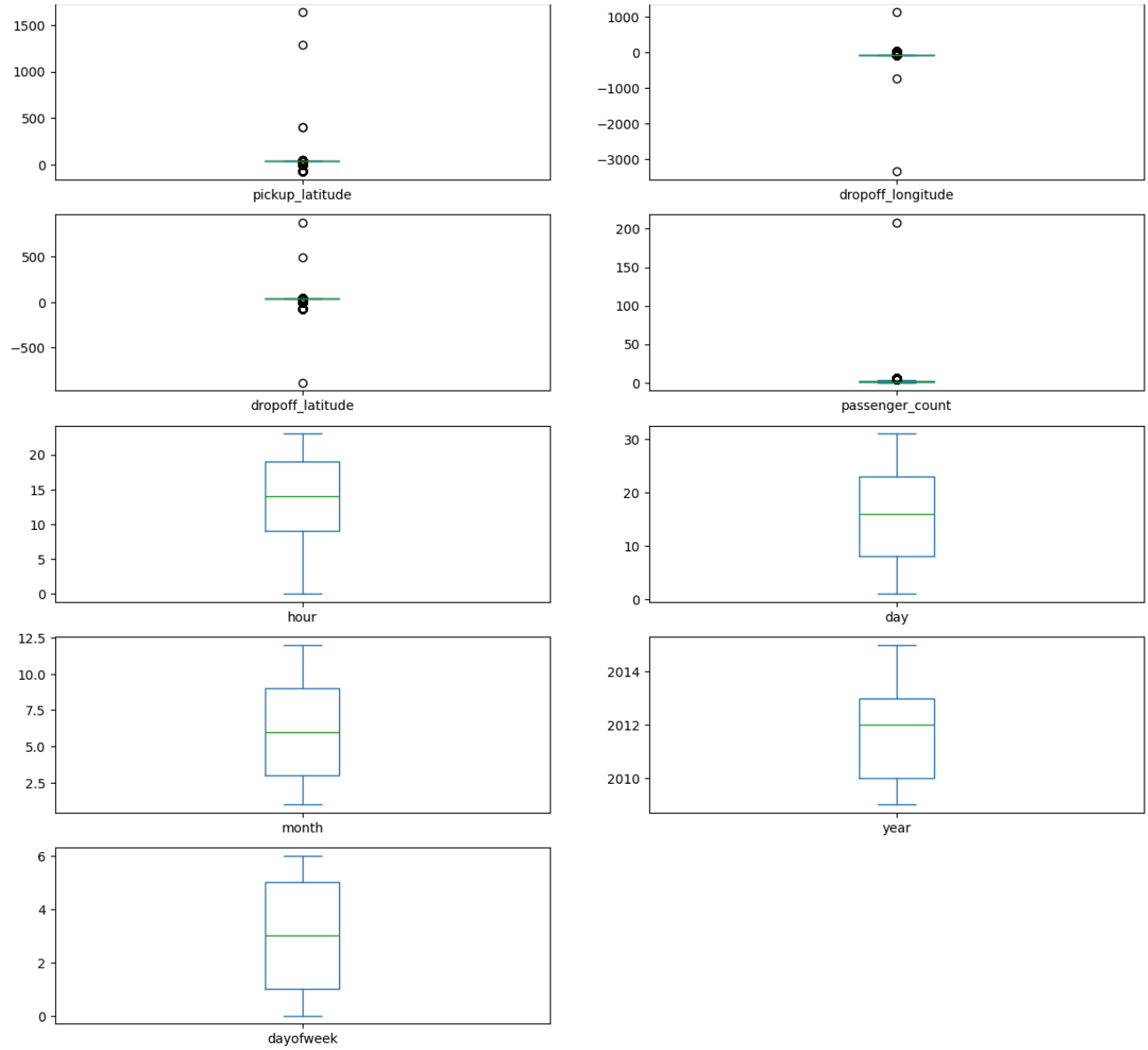
	0
fare_amount	Axes(0.125,0.786098;0.352273x0.0939024)
pickup_longitude	Axes(0.547727,0.786098;0.352273x0.0939024)
pickup_latitude	Axes(0.125,0.673415;0.352273x0.0939024)
dropoff_longitude	Axes(0.547727,0.673415;0.352273x0.0939024)
dropoff_latitude	Axes(0.125,0.560732;0.352273x0.0939024)
passenger_count	Axes(0.547727,0.560732;0.352273x0.0939024)

```
#Using the InterQuartile Range to fill the values
def remove_outlier(df1 , col):
    Q1 = df1[col].quantile(0.25)
    Q3 = df1[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_whisker = Q1-1.5*IQR
    upper_whisker = Q3+1.5*IQR
    df[col] = np.clip(df1[col] , lower_whisker , upper_whisker)
    return df1

def treat_outliers_all(df1 , col_list):
    for c in col_list:
        df1 = remove_outlier(df , c)
    return df1
```

```
df = treat_outliers_all(df , df.iloc[: , 0::])
```

```
df.plot(kind = "box",subplots = True,layout = (7,2),figsize=(15,20)) #Boxplot shows that dataset is free from outliers
```

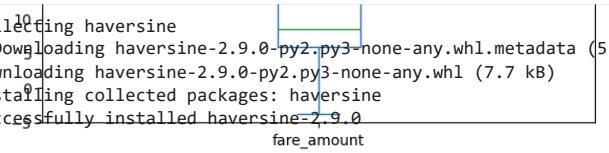


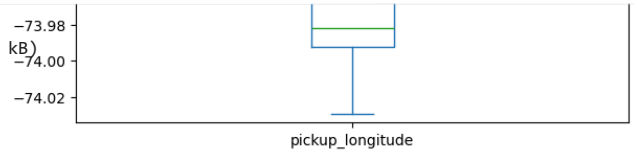
	0
fare_amount	Axes(0.125,0.786098;0.352273x0.0939024)
pickup_longitude	Axes(0.547727,0.786098;0.352273x0.0939024)
pickup_latitude	Axes(0.125,0.673415;0.352273x0.0939024)
dropoff_longitude	Axes(0.547727,0.673415;0.352273x0.0939024)
dropoff_latitude	Axes(0.125,0.560732;0.352273x0.0939024)
passenger_count	Axes(0.547727,0.560732;0.352273x0.0939024)

```
!pip install haversine
import haversine as hs  #Calculate the distance using Haversine to calculate the distance between to points. Can't use Euclad
travel_dist = []
for pos in range(len(df['pickup_longitude'])):
        long1,lati1,long2,lati2 = [df['pickup_longitude'][pos],df['pickup_latitude'][pos],df['dropoff_longitude'][pos],df['dr
        loc1=(lati1,long1)
        loc2=(lati2,long2)
        c = hs.haversine(loc1,loc2)
        travel_dist.append(c)

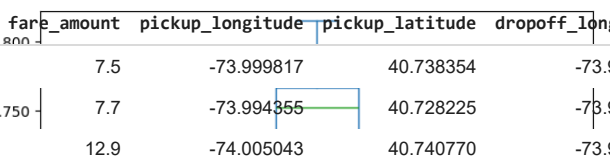
print(travel_dist)
df['dist_travel_km'] = travel_dist
df.head()
```

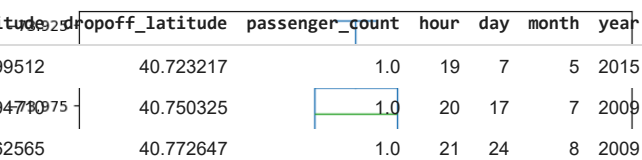
```
Collecting haversine
  Downloading haversine-2.9.0-py2.py3-none-any.whl.metadata (5.8 kB)
Downloading haversine-2.9.0-py2.py3-none-any.whl (7.7 kB)
Installing collected packages: haversine
Successfully installed haversine-2.9.0
```





	fare_amount	pickup_longitude	pickup_latitude	dropoff_longitude	dropoff_latitude	passenger_count	hour	day	month	year
0	7.5	-73.999817	40.738354	-73.999512	40.723217	1.0	19	7	5	2015
1	7.7	-73.994355	40.728225	-73.994710	40.750325	1.0	20	17	7	2009
2	12.9	-74.005043	40.740770	-73.962565	40.772647	1.0	21	24	8	2009
3	5.3	-73.976124	40.790844	-73.965316	40.803349	3.0	8	26	6	2009
4	16.0	-73.929786	40.744085	-73.973082	40.761247	3.5	17	28	8	2014






```
#Uber doesn't travel over 130 kms so minimize the distance
df= df.loc[(df.dist_travel_km >= 1) | (df.dist_travel_km <= 130)]
print("Remaining observastions in the dataset:", df.shape)
```

```
Remaining observastions in the dataset: (200000, 12)
```

```
#Finding inccorect latitude (Less than or greater than 90) and longitude (greater than or less than 180)
incorrect_coordinates = df.loc[(df.pickup_latitude > 90) |(df.pickup_latitude < -90) |
                               (df.dropoff_latitude > 90) |(df.dropoff_latitude < -90) |
                               (df.pickup_longitude > 180) |(df.pickup_longitude < -180) |
                               (df.dropoff_longitude > 90) |(df.dropoff_longitude < -90)
                                ]
```

```
df = df.drop(incorrect_coordinates.index, errors='ignore').copy()
```

```
df.head()
```

	fare_amount	pickup_longitude	pickup_latitude	dropoff_longitude	dropoff_latitude	passenger_count	hour	day	month	year
0	7.5	-73.999817	40.738354	-73.999512	40.723217	1.0	19	7	5	2015
1	7.7	-73.994355	40.728225	-73.994710	40.750325	1.0	20	17	7	2009
2	12.9	-74.005043	40.740770	-73.962565	40.772647	1.0	21	24	8	2009
3	5.3	-73.976124	40.790844	-73.965316	40.803349	3.0	8	26	6	2009
4	16.0	-73.929786	40.744085	-73.973082	40.761247	3.5	17	28	8	2014

```
df.isnull().sum()
```

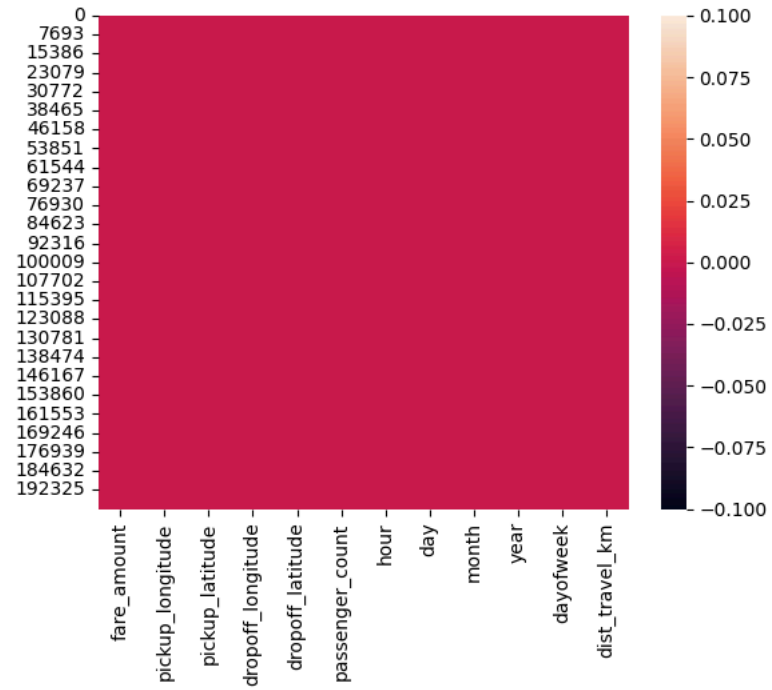


	0
fare_amount	0
pickup_longitude	0
pickup_latitude	0
dropoff_longitude	0
dropoff_latitude	0
passenger_count	0
hour	0
day	0
month	0
year	0
dayofweek	0
dist_travel_km	0

dtype: int64

```
sns.heatmap(df.isnull()) #Free for null values
```

```
<Axes: >
```




```
corr = df.corr() #Function to find the correlation
```

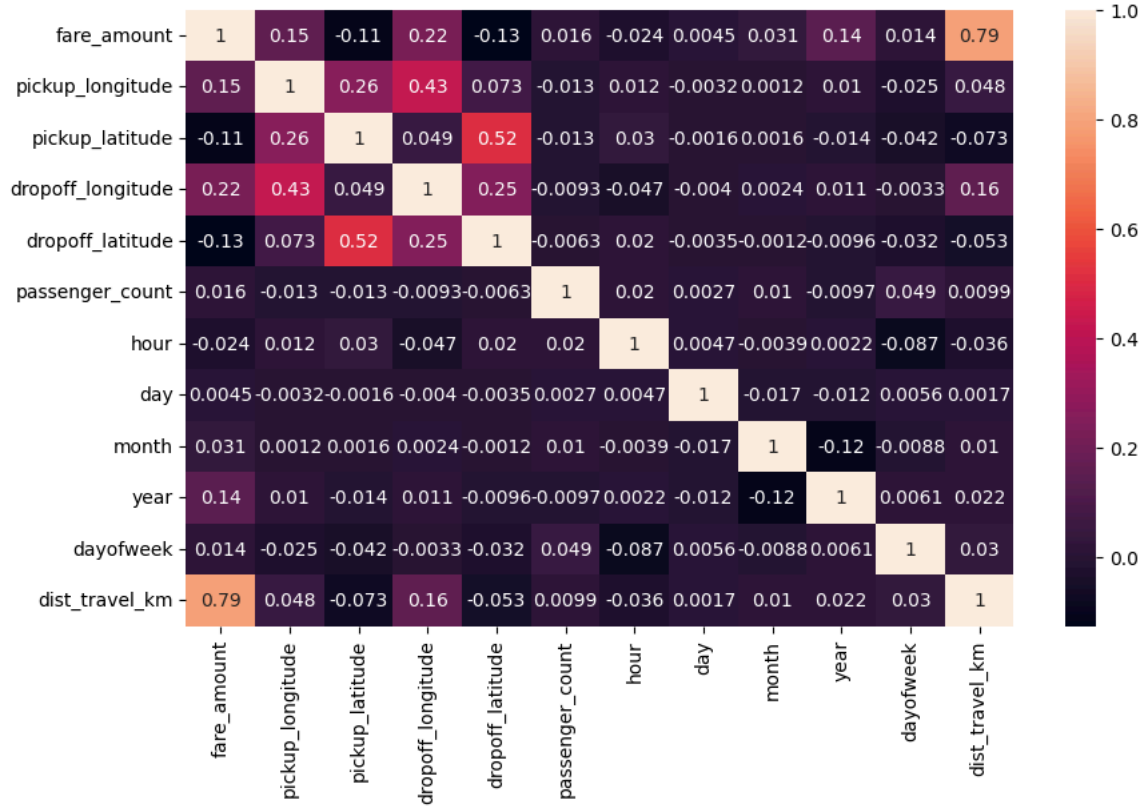
```
corr
```

	fare_amount	pickup_longitude	pickup_latitude	dropoff_longitude	dropoff_latitude	passenger_count	hou
fare_amount	1.000000	0.154069	-0.110842	0.218675	-0.125898	0.015778	-0.02362
pickup_longitude	0.154069	1.000000	0.259497	0.425619	0.073290	-0.013213	0.01157
pickup_latitude	-0.110842	0.259497	1.000000	0.048889	0.515714	-0.012889	0.02968
dropoff_longitude	0.218675	0.425619	0.048889	1.000000	0.245667	-0.009303	-0.04655
dropoff_latitude	-0.125898	0.073290	0.515714	0.245667	1.000000	-0.006308	0.01978
passenger_count	0.015778	-0.013213	-0.012889	-0.009303	-0.006308	1.000000	0.02027
hour	-0.023623	0.011579	0.029681	-0.046558	0.019783	0.020274	1.00000
day	0.004534	-0.003204	-0.001553	-0.004007	-0.003479	0.002712	0.00467
month	0.030817	0.001169	0.001562	0.002391	-0.001193	0.010351	-0.00392
year	0.141277	0.010198	-0.014243	0.011346	-0.009603	-0.009749	0.00215
dayofweek	0.013652	-0.024652	-0.042310	-0.003336	-0.031919	0.048550	-0.08694
dist_travel_km	0.786385	0.048446	-0.073362	0.155191	-0.052701	0.009884	-0.03570

Next steps: Generate code with corr | New interactive sheet

```
fig,axis = plt.subplots(figsize = (10,6))
sns.heatmap(df.corr(),annot = True) #Correlation Heatmap (Light values means highly correlated)
```

```
<Axes: >
```



Dividing the dataset into feature and target values

```
x = df[['pickup_longitude','pickup_latitude','dropoff_longitude','dropoff_latitude','passenger_count','hour','day','month','y
```

```
y = df['fare_amount']
```

Dividing the dataset into training and testing dataset

```
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test = train_test_split(x,y,test_size = 0.33)
```

Linear Regression

```
from sklearn.linear_model import LinearRegression
regression = LinearRegression()
```

```
regression.fit(X_train,y_train)
```

```
LinearRegression
LinearRegression()
```

```
regression.intercept_ #To find the linear intercept
```

```
np.float64(3676.1086843771977)
```

```
regression.coef_ #To find the linear coeeficient
```

```
array([ 2.57913661e+01, -6.81111301e+00,  2.00962146e+01, -1.83334879e+01,
        6.96941968e-02,  6.12798592e-03,  3.33880584e-03,  5.96467051e-02,
        3.71614740e-01, -3.46536917e-02,  1.84611967e+00])
```

```
prediction = regression.predict(X_test) #To predict the target values
```

```
print(prediction)
```

```
[ 9.97427504 22.1550072   9.87029109 ...  6.34332939 10.3919217
  6.73251775]
```

```
y_test
```

	fare_amount
163274	8.50
72978	22.25
114619	9.50
16360	3.70
31535	2.50
...	...
177682	5.30
173947	15.50
143685	5.70
23423	16.90
99541	7.70

66000 rows × 1 columns

dtype: float64

Metrics Evaluation using R2, Mean Squared Error, Root Mean Sqared Error